

5.1 Implementation

We have implemented *pMapper* to solve the cost minimization problem described in Eqn. 2. In this formulation, the *Arbitrator* is driven by performance goals and only arbitrates between the *Power Manager* and *Migration Manager* to find a placement that optimizes the power-migration cost tradeoff. The *Power Manager* implements the power-minimization algorithms *mPPH* and *mPP*, whereas the *Arbitrator* implements the *pMaP* algorithm to optimize the power migration tradeoff.

We use IBM Active Energy Manager [13] for monitoring power usage and EWLM [14] as the performance manager. In order to use the Active Energy Manager for monitoring, we have written a monitoring agent that is co-located with the IBM Director Server and uses network sockets to communicate with the *pMapper* framework. EWLM uses a metric called Performance Index (PI) to indicate if an application is meeting its required SLA. A PI value of 1 is achieved when the response time of the application equals the target response time as specified in its SLA. Whenever, an application fails to meet its PI or outperforms its SLA, EWLM automatically resizes the VM so that the PI value for the application reaches 1. We have implemented our *Arbitrator* to work with a workload manager independent datastructure that captures a configuration in terms of VM sizes and their placements. Hence, the *Arbitrator* uses an adapter that allow it to understand the performance characteristics and partition sizes as reported by EWLM. We have implemented *pMapper* for VMWare ESX-based platforms and hence we use VMWare Virtual Center as the *Virtualization Manager*. We use the VI API provided by VMWare ESX 3.0 to communicate with the Virtualization Manager and execute migration actions. In order to execute throttling actions, we use the IBM Active Energy Manager interface that directly communicates with the BMC via IPMI commands.

We next describe our experimental setup for the performance study.

5.2 Experimental Setup

Our experimental testbed is driven by server utilization traces obtained from the server farm of a large data center. We initially wanted to conduct our experiments on the complete implementation of *pMapper*. However, we soon realized that this would take an inordinately long time (running the experiments for Fig. 7(b) only would take 4 months as it required 60 different runs). Initially, we looked at ways to speed up the experiments but we could not do so because the data refresh rate of Active Energy Manager was 5 minutes. We realized that our implementation was not focussing on performance (which was out-sourced to EWLM) and we only needed to study the power and migration cost minimization ability of *pMapper*.

Hence, we simulated the performance infrastructure by replacing the *Performance Manager* with the application trace data. Further, once the *Arbitrator* came up with a final placement, we fed the output to a *Simulator* that estimated the overall cost of the placement. Since we had characterized the applications on the platform earlier, we could design a table-driven simulator that was accurate